TECHNICAL INTERVIEW · KNOWLEDGE BASE

Software Developer – Integration

Luxair · Node.js / TypeScript integration platform · likely technical questions & model answers

Scope in one line: integration layer handling 100M+ req/month, ~20 internal apps, 50+ third-party integrations. Emphasized in interview: **SQL (PostgreSQL + Oracle)**, **Mule → Node.js migration**, **queues / pub-sub (RabbitMQ)**, **error handling**, **resilience**. From the posting: **SOA / microservices / API design**, **Docker**, **Azure**, **CI/CD**.

1 SQL & Relational Databases HEAVILY EMPHASIZED

PostgreSQL and Oracle are both in the stack — expect portability and join/index questions.

Q Explain the different types of JOINs.

- `INNER JOIN` — only matching rows in both tables.
- `LEFT / RIGHT JOIN` — all rows from one side, `NULL` where no match.
- `FULL OUTER JOIN` — all rows from both, matched where possible.
- `CROSS JOIN` — Cartesian product. `SELF JOIN` — a table joined to itself (e.g. employee→manager).

Q What is an index, and when does it hurt rather than help?

A B-tree index speeds up lookups, joins and `ORDER BY` by avoiding full scans. Cost: it slows `INSERT/UPDATE/DELETE` (the index must be maintained) and consumes storage. Indexes are wasted on low-cardinality columns (e.g. a boolean) and on tables small enough that a scan is cheaper. Composite index column order matters — leftmost-prefix rule.

Q `WHERE` vs `HAVING`?

`WHERE` filters rows *before* aggregation; `HAVING` filters *after* `GROUP BY` on aggregate results. Filter in `WHERE` whenever possible — it reduces the rows that must be grouped.

Q How would you find and fix a slow query?

Run `EXPLAIN ANALYZE` (Postgres) / `EXPLAIN PLAN` (Oracle). Look for sequential scans on large tables, bad row estimates, and expensive nested loops. Fixes: add/adjust indexes, rewrite correlated subqueries as joins, avoid `SELECT *`, ensure statistics are current (`ANALYZE` / `DBMS_STATS`), and select only needed columns to enable index-only scans.

Q What are ACID properties?

Atomicity (all-or-nothing), **C**onsistency (constraints always hold), **I**solation (concurrent txns don't interfere), **D**urability (committed data survives crashes). Relevant when you decide what belongs in a single transaction during integration writes.

Q PostgreSQL vs Oracle — practical differences you'd watch for?

Topic	PostgreSQL	Oracle
Auto IDs	`SERIAL` / `IDENTITY`	Sequences / `IDENTITY`
Pagination	`LIMIT / OFFSET`	`FETCH FIRST n ROWS` / `ROWNUM`
Empty string	Distinct from `NULL`	Treated as `NULL`
String concat	`||` or `CONCAT`	`||` or `CONCAT`

> **Integration angle:** when the same query must run against both, avoid vendor-specific syntax or isolate it behind a data-access layer — this is exactly the kind of portability concern a migration team cares about.

Q Transaction isolation levels — name them and the anomaly each prevents.

Read Uncommitted → dirty reads possible; Read Committed → prevents dirty reads (Postgres default); Repeatable Read → prevents non-repeatable reads; Serializable → prevents phantom reads, behaves as if txns ran sequentially. Higher isolation = more locking/contention.

Q How do you prevent SQL injection?

Parameterized / prepared statements — never string-concatenate user input. In Node this means `pool.query('... WHERE id = $1', [id])` (pg) rather than template literals. ORMs/query builders parameterize by default; be careful with raw-query escape hatches.

2 Mule → Node.js Migration & Integration Architecture THEIR CURRENT PROJECT

They're actively replacing a Mule ESB with Node.js; microservices was floated as the likely direction.

Q What does an ESB like Mule do, and why migrate off it?

An Enterprise Service Bus is a central hub that connects systems: routing, message transformation, protocol bridging, and orchestration via pre-built connectors. Teams move off it for cost/licensing, to escape a heavyweight central bottleneck, for finer scaling and deploy independence, and to use a broader hiring pool and ecosystem (Node/TS) instead of a specialized ESB skillset.

Q ESB (hub-and-spoke) vs microservices — the core trade?

ESB centralizes integration logic in one bus — simple to see, but a single point of change and failure. Microservices decentralize: each service owns its integration and can scale/deploy independently, at the cost of more operational complexity (service discovery, distributed tracing, network failure handling).

Q How would you approach migrating a live integration incrementally?

- **Strangler Fig pattern:** put a facade/proxy in front, route one flow at a time to the new Node service, shrink Mule gradually — no big-bang cutover.
- Start with a low-risk, well-understood flow to build confidence and tooling.
- Run old and new in parallel and compare outputs before switching traffic.
- Keep contracts stable so downstream consumers don't notice the swap.

Q SOA vs microservices — aren't they the same?

Both are service-oriented. SOA is coarser-grained, often shares a bus and sometimes a database, enterprise-wide reuse focus. Microservices are finer-grained, independently deployable, database-per-service, "smart endpoints / dumb pipes." Microservices are essentially a stricter, more decentralized take on SOA principles.

Q How do services communicate — and when sync vs async?

Sync (REST/gRPC): caller waits, simple, good when you need an immediate answer — but couples availability (if the callee is down, you're down). **Async** (queues/events): fire-and-forget, decouples services, absorbs load spikes, survives temporary outages — but adds eventual consistency and harder debugging. Integration platforms lean async for third-party calls that can be slow or flaky.

3 API Design CORE ROLE

Q What makes a good REST API? Map the main HTTP methods and status codes.

- `GET` read (safe, idempotent), `POST` create, `PUT` replace (idempotent), `PATCH` partial update, `DELETE` remove (idempotent).
- 2xx success (`200` , `201` created, `204` no content); 4xx client error (`400` , `401` unauth, `403` forbidden, `404` , `409` conflict, `429` rate-limited); 5xx server error (`500` , `502/503` upstream/unavailable).
- Noun-based resource URLs, plural, no verbs: `/orders/123` not `/getOrder` .

Q What does idempotency mean and why does it matter for integrations?

The same request applied multiple times has the same effect as once. Critical because networks retry — if a client re-sends a payment call after a timeout, an idempotent endpoint (often keyed by an `Idempotency-Key` header) prevents a double charge. This is a very likely question given a retry-heavy integration platform.

Q REST vs GraphQL vs gRPC — when each?

REST: default, cache-friendly, ubiquitous for public/third-party APIs. GraphQL: client picks exact fields, avoids over/under-fetching, good for varied frontends. gRPC: binary, fast, strongly typed contracts — great for internal service-to-service, less so for browsers/public APIs.

Q How do you version and evolve a public API without breaking consumers?

Version via URI (`/v1/`) or header. Only make additive changes within a version (new optional fields), never remove/rename existing ones. Deprecate with clear timelines and communication. With 50+ integrations, backward compatibility is a first-class concern.

Q How do you secure an API?

AuthN with OAuth2 / JWT or API keys; AuthZ checks per resource; HTTPS everywhere; input validation; rate limiting / throttling (`429`) to protect against abuse and cascading load; never leak internal error details or secrets in responses.

4 Queues & Publish–Subscribe (RabbitMQ) THEY ASKED DIRECTLY

Interviewers explicitly asked about queueing / pub-sub experience, naming RabbitMQ.

Q Why use a message queue in an integration platform?

- **Decoupling:** producer and consumer don't need to be up at the same time.
- **Load levelling:** the queue buffers spikes so a slow downstream/third-party isn't overwhelmed.
- **Resilience:** if a consumer crashes, messages wait rather than being lost.
- **Async work:** return to the caller fast, process heavy work in the background.

Q Explain RabbitMQ's model: exchange, queue, binding, routing key.

Producers publish to an **exchange**, not directly to a queue. The exchange routes to **queues** based on **bindings** and a **routing key**. Types: `direct` (exact key match), `topic` (wildcard patterns like `order.*`), `fanout` (broadcast to all bound queues — this is pub/sub), `headers` .

Q Point-to-point queue vs publish–subscribe?

In a work queue, each message goes to exactly *one* consumer (competing consumers share the load). In pub/sub (fanout), each subscriber gets its *own copy* — one event, many independent reactions. "Order placed" → one queue charges the card, another sends email, another updates analytics.

Q How do you guarantee a message isn't lost?

Publisher confirms, durable queues + persistent messages (survive broker restart), and manual consumer **acks** — the broker only removes a message after the consumer acknowledges successful processing. If the consumer dies mid-work, the un-acked message is redelivered.

Q What's a dead-letter queue?

A separate queue where messages land after they fail processing repeatedly (or expire / are rejected). It stops a "poison" message from being retried forever and blocking the queue, and lets you inspect, alert on, and reprocess failures. Very relevant to the error-handling and resilience themes below.

Q Kafka vs RabbitMQ — one-line distinction?

RabbitMQ is a traditional broker: routes messages, deletes after ack — great for task distribution and RPC-style work. Kafka is a distributed log: retains an ordered event stream that many consumers replay at their own offset — great for high-throughput event streaming and event sourcing.

5 Error Handling THEY ASKED DIRECTLY

They asked how systems you'd worked on handled errors and which approaches you used.

Q How do you handle errors in async Node.js code?

- `async/await` wrapped in `try/catch`; for Express, forward to centralized error middleware (or use a wrapper so rejected promises reach it).
- Always handle promise rejections — an unhandled rejection can crash the process.
- Distinguish **operational errors** (expected: bad input, timeout, 3rd-party down → handle gracefully) from **programmer errors** (bugs → fail fast, fix). This distinction is a strong thing to say out loud.

```
// centralized Express error handler
app.use((err, req, res, next) => {
  logger.error({ err, reqId: req.id });
  const status = err.statusCode || 500;
  res.status(status).json({ error: err.publicMessage || 'Internal error' });
});
```

Q A third-party integration fails intermittently. How do you handle it?

Retry with exponential backoff + jitter — but only for transient/retryable errors (timeouts, `503`, `429`), never for `400/401`. Cap the retries, then fail over to a fallback or dead-letter the message. Make the operation idempotent so retries are safe. Add a timeout on every outbound call so a hanging third party can't block you.

Q How should errors surface to API callers vs internal logs?

To callers: correct status code + a stable, safe error message/code — no stack traces, SQL, or internal hostnames. Internally: structured logs (JSON) with a correlation/request ID so one request can be traced across services, plus enough context to reproduce. Never swallow errors silently.

Q What is graceful degradation?

When a dependency fails, degrade instead of collapsing — serve cached/stale data, return a partial response, or disable a non-critical feature while the core keeps working. Better to lose one feature than the whole request.

6 Resilience EXPLICITLY MENTIONED

"Resilience" was named directly — be ready to define the patterns, not just the word.

Q Explain the Circuit Breaker pattern.

Wraps calls to a flaky dependency. **Closed** = calls pass through. After a failure threshold it trips to **Open** = calls fail fast immediately (no waiting on a dead service), giving it time to recover. After a cooldown it goes **Half-Open**, letting a few trial calls through; success → Closed, failure → Open again. Stops one failing service from cascading and exhausting your resources. (In Node: e.g. `opossum`.)

Q Which resilience patterns would you combine on an outbound integration call?

- **Timeout** — never wait indefinitely.
- **Retry + exponential backoff + jitter** — for transient failures, jitter avoids thundering-herd retries.
- **Circuit breaker** — stop hammering a service that's clearly down.
- **Bulkhead** — isolate resource pools so one saturated dependency can't starve the rest.
- **Fallback / dead-letter** — a defined path when all else fails.

Q How do you make the platform observable / know when something breaks?

Three pillars: **metrics** (latency, error rate, throughput — e.g. Prometheus), **logs** (structured, correlated by request ID), **traces** (follow one request across services — OpenTelemetry). Add health-check endpoints (`/health`, readiness vs liveness) and alerting on error-rate/latency thresholds. You can't be resilient to failures you can't see.

Q How do you scale for 100M+ requests/month reliably?

Stateless services behind a load balancer (scale horizontally), caching for hot reads, async queues to smooth spikes, connection pooling to the DB, and rate limiting to shed abusive load. Design for failure: assume any instance or dependency can drop and ensure no single one takes the system down.

7 Docker, Azure & CI/CD FROM THE POSTING

Q Why containers, and what problem does Docker solve?

Packages the app with its exact dependencies so it runs identically across dev, CI and prod — kills "works on my machine." Lighter than VMs (shares the host kernel), starts in seconds, and is the natural deploy unit for microservices. Know: `image` vs `container`, `Dockerfile`, layers, and `docker-compose` for multi-service local setups.

Q What's a multi-stage Docker build and why use it for Node?

Build/compile (incl. TypeScript, dev dependencies) in one stage, then copy only the built output + production deps into a slim final image. Result: smaller, faster, more secure images with no build tooling shipped to prod.

Q What Azure services would touch this kind of workload?

Compute: **App Service** or **Azure Container Apps / AKS** for containers, **Azure Functions** for event-driven bits. Messaging: **Service Bus** (enterprise queues/pub-sub — the Azure analogue to RabbitMQ) and **Event Grid/Hubs**. Also **Azure SQL / DB for PostgreSQL**, **Key Vault** for secrets, **Application Insights** for monitoring. Honesty tip: be frank about depth and lean on transferable cloud concepts.

Q CI vs CD — and what's in a good pipeline?

CI: every push builds and runs the test suite automatically, catching breakage early. **CD**: automated delivery/deploy of passing builds. A typical pipeline: lint → unit/integration tests → build image → security scan → deploy to staging → (approval) → prod, with the ability to roll back. Enables small, frequent, low-risk releases.

Q How do you deploy without downtime?

Rolling, blue-green, or canary deployments behind a load balancer, gated by readiness health checks so traffic only hits instances that are actually ready. Keep DB migrations backward-compatible (expand → migrate → contract) so old and new versions can run side by side during rollout.

8 Node.js / TypeScript Quick-Fire FUNDAMENTALS

Q How does Node handle concurrency with a single thread?

Single-threaded **event loop** with non-blocking I/O: I/O is offloaded (libuv thread pool / OS), and callbacks run when ready — so one thread serves many connections. CPU-bound work blocks the loop, so offload it to `worker_threads` or a separate service. Scale across cores with the `cluster` module or multiple container instances.

Q What does TypeScript add over JavaScript here?

Static typing catches errors at compile time, self-documents contracts (great for API request/response shapes and service boundaries), and makes refactoring across a large integration codebase far safer. Compiles to plain JS; types are erased at runtime.

Q `==` vs `===`, and what are Promises?

`===` strict equality, no type coercion — always prefer it. A Promise represents an eventual async result (`pending → fulfilled/rejected`); `async/await` is syntactic sugar over Promises. Use `Promise.all` for parallel independent async calls, `allSettled` when you need every result regardless of individual failures.

> **Closing tip for the room:** the interviewers care about integration, migration and reliability — where you don't know a specific tool (e.g. deep Azure or Oracle internals), name the equivalent concept you *do* know and how you'd map it. Demonstrating transferable reasoning about resilience, async messaging and API contracts lands better than bluffing specifics.